SAVE

GIT-VISTA · MILESTONE 3 · ISSUE #78

An undo button for Git

Save points, and second chances.

Git already writes down everything it does. The problem is that reading that record needs magic words you have to know in advance, and some of it disappears after 30 days. This is about turning that hidden record into a list you can actually look at — and, where it is still safe, a button that puts things back.

WHAT THIS DOCUMENT DECIDES

The hard part is already built

Two designs were written for this. The one that won is the boring one: the app *already* writes down every action in its own notebook, including the ones it refused to do. So this job is mostly about **reading** that notebook, not building a new one. That makes it far smaller than it looked.

Refusals count as history too

"I would not do that, your copy is out of date" is an answer, and it gets written down like any other. A history that only showed successes would quietly hide the most interesting days.

Never offer an undo you cannot stand behind

Before showing an *Undo* button, the app checks — right then, not from memory — that the old version is still reachable. If the check itself cannot run, that is **not** the same as "safe", and the button stays hidden. "Could not tell" must never read as "yes".

One piece has no design at all

Setting work aside half-finished (#77) still has nothing written for it. The agent that was supposed to design it failed five times and returned nothing, so this document says so plainly instead of pretending otherwise.

MILESTONE	M3 — Parallel Work & Recovery
THIS DOC	the decision behind issue #78, before any code
STATUS	designed, nothing built yet
STILL OPEN	#77 stash has no design

Status: Design spec, pre-ADR — for Codex adversarial review before an ADR is filed under `docs/adr/`. **Extends:** #78, building on the durable-write mechanism M1.09 (#62) shipped (`durable.rs`) and the live-recompute posture the activity feed already established (#327). **Supersedes:** nothing yet — this is the reconciliation of two independently-produced designs (Approach A, durable-first; Approach B, derive-from-git-first), grafted into one.

Context

Two designs came back for the same feature — a browsable history of what git-vista itself did, with live-verified recovery offers. Both start from the same grounding (reproduced faithfully in both): a durable SQLite journal (`durable.rs`) already records every admitted operation at admission and again at its terminal transition, keyed by idempotency key, carrying the closed `GitOperation`, its hash, repository/worktree tokens, generation, and a static `RecoveryStrategy`; a per-repo `journal.jsonl`/`refs.json` pair already backs the *activity feed* (event-centric, tool-attributed, includes external git); a recovery-ref pin (`refs/git-vista/recovery/<id>`) already survives `git gc` by being a real ref, written from inside the mutation guard immediately before `execute`.

Approach A treats the SQLite table as the source of truth and adds a read path plus live classification on top. Approach B treats git's own reflog as the source of truth and shrinks — but does not eliminate — a sidecar for what reflog structurally cannot carry (deletions, refusals, operation identity). Task item 2 additionally asked me to reconcile "the stash design" against the winner; no third design document was supplied to me, and this spec says so plainly rather than inventing content for one — see Reconciling stash below.

I independently verified the load-bearing citations in both designs against the tree at `main` before writing this (see Corrections for what didn't hold up) and read three files neither design's grounding opened:

`planner.rs`'s admission/validation ordering, `operation.rs`'s `OperationState` doc, and `catalog.rs`'s `Catalog::resolve`. Those three reads are what decide this.

Decision

Approach A (durable-first) wins. Build the Recovery Center's read path over `durable.rs`'s `operations` table: one new non-mutating query, one live classification function, one write endpoint that re-derives and gates on equality before executing. Do not build a parallel reflog-derived history; the activity feed already owns that job and stays untouched.

The discriminating finding — neither design's own grounding had it

`plan_and_execute_tracked` calls `crate::operations::admit` *before* `validate()`/`enforce_fresh()` run (`planner.rs:198`, confirmed by the doc comment at `planner.rs:6002`ish stating admission precedes validation). Both gates live *inside* `plan_and_execute_in`, called from the detached task `plan_and_execute_tracked` spawns, and both return early as `Result<(), (StatusCode, String)>` propagated straight out as the function's own `(StatusCode, String)` return (`planner.rs:330-337`, the `if let Err(refused) = validate(&plan) { return refused; }` / `enforce_fresh` pair). That return value becomes `status`/`message` in the spawned task (`planner.rs:236`), then `terminal = handle.terminal_status(status, &message, generation)`, then **unconditionally** `crate::durable::persist(durable_key, terminal.clone()).await` (`planner.rs:264`) — the same `persist` call that runs on a genuine success.

`OperationState::Failed`'s own doc says why this is not an accident: *"A refusal is an outcome: 'your commit was refused as stale' is an answer, where a lost connection is not."* (`crates/git-vista-protocol/src/operation.rs:94-97`).

So the durable table already durably records refusals and failures, today, with no code change. This kills Approach B's central justification for widening `journal.jsonl` (B's model §3: *"Failures and refusals write nothing to git... A history feed built purely from git's own records would show only the operations that succeeded"*). That sentence is true of git's reflog — reflog genuinely can't carry a refusal — but B's own remedy (a new `OperationHistoryRecord` written on every terminal transition including `Refused`/`Failed`) reinvents, in a second sidecar, something the SQLite table already is. B never opened `planner.rs` far enough to see the unconditional second `persist` call, and its own grounding (§3 above, "What would #78 have to hook into?") stopped at "the chokepoint already exists" without checking whether refusals flow through it. They do.

The narrower, honest form of the claim (Approach A's summary did not state this precision either): this is true only for operations that reach `admit()`. A request that fails per-handler validation, a read-only-clone refusal, or `Admission::Conflict` (a replayed idempotency key against a different hash, `planner.rs:206-213`ish) returns *before* `admit()` and leaves no row — those are not history under this design, and this spec does not claim they are. `refuse_if_git_busy` (`planner.rs:325-327`, "outside git holds the index") is the contrast case: it runs *after* `admit()`, inside `plan_and_execute_in`, so it *is* captured by the same unconditional `persist`.

What is grafted from Approach B, named explicitly

1. **The three-way fail-closed shape for a live check that could not run**, distinct from a live check that ran and returned a definite negative. B's `RecoveryAvailability::Unverifiable` names a real gap in A's original `RecoveryClass::Expired` (which conflated "the pin doesn't resolve" with "the check itself couldn't be attempted"). This codebase already has the exact precedent, and it's a stronger citation than either design used: `revert_offer_established` (`crates/git-vista-server/src/activity.rs`, ~lines 335-363) collapses three distinct failure shapes — HEAD doesn't resolve, resolving it couldn't even run, or `revert_would_conflict` itself errored — to the same `false`, with the comment *"None of these is 'no*

conflict' — they're 'no fact', and a fact we don't have is never grounds to offer" (activity.rs:350-354), and `revert_would_conflict`'s own doc: *"'couldn't tell' must never read as 'safe to offer'"* (activity.rs:314). I add `RecoveryClass::CheckFailed` to A's enum for this (see Model), keeping A's shape (only `Offered` carries an `UndoAction`) rather than importing B's separate `RecoveryAvailability` type.

2. **The two gc-expiry numbers**, labeled correctly as *environment-verified, not code-verified*: `gc.reflogExpire` defaults to 90 days, `gc.reflogExpireUnreachable` (the shorter, more relevant one — it governs objects unreachable from the current tip, exactly what a force-delete/reset leaves) defaults to 30 days, confirmed via `man git-gc`/`git-config` on this box, git 2.43.0, no local overrides. These strengthen A's own risk note about the recovery-ref pin being the only thing standing between a `RecreateTag`/`ResetRef` offer and permanent loss.
3. **Two negative findings, both re-verified by me independently, not merely restated**: `ORIG_HEAD` has zero production references anywhere in the tree (`grep -rn "ORIG_HEAD"` across every `.rs` file outside `target/`, zero hits — I ran this myself, not trusting B's unverified claim of the same) — correctly unused, stays unused. And no production code path invokes `git stash`: it appears only in `sandbox/dispatch.rs`'s local-subcommand allowlist test (line 78) and as a UI glyph in `git-vista/src/icons.rs` (lines 15, 31, 85, 121, 151, 195) — confirmed by me via the same greps B described.

What A itself needed corrected, not grafted from B — see Corrections.

Model

Storage: extend the existing `operations` table (`durable.rs:113-146`), do not add a second store.

```
ALTER TABLE operations ADD COLUMN recovers_operation TEXT; -- nullable C
CREATE INDEX idx_operations_history ON operations(accepted_at, id);
CREATE INDEX idx_operations_recovers ON operations(recovers_operation) W
```

`recovers_operation` is set only on a row that is *itself* the executed recovery of an earlier operation, never backfilled onto the original row — "was X recovered" is a read-time lookup (`WHERE recovers_operation = X AND state = 'succeeded'`), not a mutable flag, so a terminal row stays functionally immutable outside the one documented startup close-out path (`recover_blocking`, `durable.rs:510-528`).

`mint_id()` mints 16 random bytes, hex-encoded (`operations.rs:548-553`) — **not time-ordered**. Pagination must be keyset on `(accepted_at, id)`, never `id` alone.

```
// crates/git-vista-server/src/recovery_center.rs (new server-side modul
// deliberately NOT crates/git-vista-protocol/src/history.rs: that file
// already exists, already exports HistoryFrame/HistoryPage for the M1.1
// paged-commit-graph feature (#63), and is unrelated. See Corrections.)

pub(crate) struct HistoryPage {
    pub entries: Vec<HistoryEntry>,
    pub next_cursor: Option<HistoryCursor>,
}
pub(crate) struct HistoryCursor { pub accepted_at: UnixSeconds, pub id:

pub(crate) struct HistoryEntry {
    pub id: OperationId,
    pub operation: GitOperation,
    pub state: OperationState,      // Succeeded | Failed only — is_term
    pub accepted_at: UnixSeconds,
    pub ended_at: UnixSeconds,      // always Some — row is terminal
    pub status: Option<u16>,
    pub message: Option<String>,
    pub repository: RepositoryToken,
    pub worktree: WorktreeToken,
    pub recovers: Option<OperationId>,
    pub recovery: RecoveryClass,    // LIVE, never served from stored re
}

pub(crate) async fn list_operations(
    repository: RepositoryToken,
    state: HistoryStateFilter,      // Succeeded | Failed | Any
    cursor: Option<HistoryCursor>,
    limit: u32,                     // server-clamped
) -> Result<HistoryPage, DurableError>;
```

```
/// What can be done about one past operation's recovery, RIGHT NOW. Onl
/// `Offered` carries an `UndoAction` — no arm of this enum other than
/// `Offered` can produce one, so an exhaustive match cannot construct c
/// forward an undo for anything else. This is a compile-time property:
/// proven by the type checker refusing a build that moves `undo` onto a
/// shared field, not by a test going red (see the invariant table).
#[serde(tag = "recovery_class", rename_all = "snake_case")]
pub enum RecoveryClass {
    NoneNeeded,                                          // RecoveryStra
    AlreadyCurrent,                                       // needed recc
    Offered { undo: UndoAction, label: String, warn_pushed: bool },
    Expired { reason: RecoveryExpiredReason },             // definite r
    /// Grafted from Approach B, named explicitly (see Decision, graft #
    /// the live check could not even run — sandboxed git spawn failed,
    /// the repository token no longer resolves in the catalog (see belc
    /// Distinct from Expired: this is "no fact", never "no". Mirrors
    /// revert_offer_established's three-way fail-closed collapse
    /// (activity.rs:335-363) and revert_would_conflict's own doc,
    /// "'couldn't tell' must never read as 'safe to offer'" (activity.r
    CheckFailed { detail: CheckFailedReason },
    KnownNotWired { strategy: RecoveryStrategy },          // real recov
    Unsupported { reason: UnsupportedReason },
}

pub enum RecoveryExpiredReason {
    PinMissing,          // refs/git-vista/recovery/<id> no longer resol
    ObjectUnreachable,    // ref resolves, object doesn't (should be imp
    TargetRefGone,        // the branch/tag name the strategy would move
    NameReused,           // the name now points at something else — rec
}

pub enum CheckFailedReason {
    GitSpawnFailed,
    /// The row's RepositoryToken/WorktreeToken no longer resolves via
    /// Catalog::resolve (catalog.rs:188) — the in-memory catalog is
    /// rebuilt at startup by rescanning configured roots and is documer
    /// to return None for any id it does not hold
```

```
    /// (catalog.rs: "resolve_returns_none_for_an_unknown_id" test,
    /// catalog.rs:547-557). A history row can therefore outlive the
    /// registration of the repository it names — moved, deregistered, c
    /// simply not yet rescanned this session — and classify_recovery ha
    /// no path to run git against. This is a real gap; see Open Questic
    RepositoryNotRegistered,
}

pub enum UnsupportedReason {
    EffectLeftTheRepository,   // push, ForcePublish::WithLease
    NeverJournaled,            // test-repo reset
    NeverInObjectDatabase,     // delete-untracked-paths, #219
    NoRecoverableHandle,       // RecoverableIfStaged — no ref/commit to
}

/// Live-verified against the repo — refs/git-vista/recovery/<id> and (f
/// ResetRef/RecreateBranch/RecreateTag) the live target ref — never ref
/// Called once per HistoryEntry on a page: bounded by page size, same c
/// shape as undoables()'s one-precheck-per-menu-open.
async fn classify_recovery(
    repo: &Path,
    operation_id: &OperationId,
    strategy: Option<&RecoveryStrategy>,
) -> RecoveryClass;
```

Executing a recovery reuses the undo pipeline, not a fork of it:

```
// activity.rs's undo() (activity.rs:386-455) inlines UndoAction -> GitO
// validation, then plan_and_execute. Factor the match arms out so both
// existing endpoint and the new one share it:
async fn undo_action_to_operation(repo: &Path, action: UndoAction)
    -> Result<GitOperation, (StatusCode, String)>;

// POST /api/operations/{id}/recover , body { action: UndoAction }
async fn recover_operation(id: OperationId, Json(claimed): Json<UndoActi
    -> (StatusCode, String) {
    // 1. Load the durable row; 404 unknown, 400 not-Succeeded.
    // 2. classify_recovery(repo, &id, row.recovery.as_ref()).await — LI
    //    never the client's cached class from an earlier page load.
    // 3. 409 unless the result is RecoveryClass::Offered { undo, .. } A
    //    undo == claimed structurally. THIS comparison is the actual
    //    enforcement point for "unsupported recovery is never labeled
    //    undo" at the write boundary — the enum shape keeps the server
    //    honest; this equality keeps a stale or hand-crafted client rec
    //    from mattering. Same posture as #145's staleness gate, one lay
    // 4. op = undo_action_to_operation(repo, claimed)?;
    // 5. thread recovers = Some(id) through admit() (new optional field
    //    additive on OperationStatus — safe because its Deserialize doe
    //    not deny_unknown_fields, operation.rs's own doc: "an older cli
    //    must keep parsing it when a later protocol adds a field").
}
```

Surfaces

HTTP: - `GET /api/operations/history?repository=<token>&state=<succeeded|failed|any>&before=<cursor>&limit=<n>` → `HistoryPage`. `no-store`, matching `/api/undoables/{id}`'s posture — a live-recomputed view must never be cached across a mutation. - `GET /api/operations/{id}/recovery` → single `RecoveryClass`, for a detail pane or a pre-click refresh (a list row may be minutes stale by the time it's read). - `POST /api/operations/{id}/recover` — as above. 404 unknown id, 400 not-Succeeded, 409 classification mismatch.

UI: an exhaustive match over `RecoveryClass` with no default arm — only `Offered` has an `undo` in scope to bind a button to; every other arm renders explanatory text (`label`/`reason`), never a clickable control, because there is no `UndoAction` value reachable from that arm. `warn_pushed` reuses the existing "undo never force-pushes" confirm-dialog copy.

Scoped to the currently selected `RepositoryToken` for the first cut (ADR 0003: repository is always an opaque token). See Open Questions — cross-repository browsing is deferred, not ruled out, and the `CheckFailedReason::RepositoryNotRegistered` arm above means the read path must handle an unresolved token regardless of which scope wins.

Alternatives considered

Approach B, derive-from-git-first, in full, is worth recording because its weaknesses are precisely where recovery matters most. B proposed treating reflog as primary and shrinking (not eliminating) a sidecar to what reflog structurally can't carry — deletions (reflog dies with the ref that owned it, `reflog.rs:10-12`), operation identity (one action can write multiple reflog lines, or none), and refusals. B's own honest framing: *"weakest exactly where recovery matters most, and strongest exactly where it matters least"* — a commit still at a tip needs no sidecar; a force-deleted branch past `gc.reflogExpireUnreachable`'s 30-day window has already lost both git's record and, once pruned, the content.

Rejected for three reasons, in order of weight: 1. **It duplicates a store that already exists and already has full coverage.** B's proposed `OperationHistoryRecord`/`append_history`, written on every terminal transition including `Refused`/`Failed`, is exactly what `durable.rs`'s SQLite table already is (see Decision). B never engaged with this because its own grounding stopped one file short. 2. **Scope mismatch.** The Recovery Center is operation-centric — "what did *this app* do, and can *this app* put it back" (a phrase both designs used, correctly). Reflog is ref-centric and tool-agnostic; the activity feed

already exists precisely to answer "what moved, by whom, including outside git" from reflog + journal + snapshot-diff. Rebuilding a second reflog-derived view for an app-operations question duplicates a served purpose rather than filling a gap. 3. **B's own proposed home for its new type doesn't exist as described.** B specified "New protocol type… in a new `crates/git-vista-protocol/src/history.rs` module." That file is not new: it already holds `HistoryFrame<R>`/`HistoryPage<R,E,S>` — the generic paged-commit-graph transport shapes for M1.10 (#63), completely unrelated to operation history. This is a real error in B, not a nitpick — it would have collided a new domain type into a file whose own module doc says it "declares only the transport shape, never the domain types that fill it in."

What B got right and where it survives in the winning design is enumerated in Decision's graft list. Nothing in B's model is thrown away silently — every citation of it in this document is either grafted or explained as rejected.

Reconciling stash — there is no stash design

Task item 2 asked me to reconcile "the stash design" against the winner. No third design document was supplied to me; I did not fabricate one. What exists is Approach B's own treatment of `git stash` as a candidate recovery-data source (its model §5), and I independently re-ran its greps rather than trusting them: `git stash` appears in exactly two places in the tree — a local-subcommand allowlist *test* (`sandbox/dispatch.rs:78`, confirming `git stash`/`git reflog` classify as `NetworkNeed::Local`) and a UI glyph set (`git-vista/src/icons.rs`, six lines, all cosmetic). No planner, handler, or `GitOperation` variant invokes `git stash` anywhere. `ADR 0001` already rules stash out deliberately, at the identity/generation layer, with its own words: *"Stash and notes are deliberately out of the current input set"* (`docs/adr/0001-repository-generation.md:123`).

The reconciliation that survives either reading of the task instruction: **stash cannot appear in this surface today, because no `RecoveryStrategy` variant can name one and no operation**

creates one. If a future milestone adds a stash-creating `GitOperation`, its recovery lands in `RecoveryClass::KnownNotWired` by construction the moment it's given a `RecoveryStrategy` with no matching `UndoAction` — the type already has the slot; nothing in this design needs to change to accommodate it later.

Consequences

Reuse over rebuild, at every seam. The durable write-before/after mechanism (the actual M1.09 asset) is untouched; the recovery-ref pin is the sole live-verification anchor for gc-survival, reused not reimplemented; the undo execution path (`UndoAction`/`plan_and_execute`) is factored, not forked. New runtime surface is small: one column, two indexes, one query function, one classification function, one endpoint, one additive field on `admit()`/`OperationStatus`.

Append-only-in-effect rows. Deriving "was this recovered" by lookup rather than a mutable column costs one extra indexed query per detail view but keeps `durable.rs`'s existing "terminal rows are functionally immutable outside startup close-out" property true instead of adding a second mutation path to reason about.

Risks, stated plainly, in the ADR-0058 spirit:

1. **The equality gate in `recover_operation` step 3 is the single highest-risk point.** If a later refactor ever treats the request body as authoritative ("the UI already validated it"), the type-system guarantee this design exists to provide becomes decorative — a stale or hand-crafted `UndoAction` could execute against a world that has moved past `Offered`. Same seriousness as #145's plan-staleness gate.
2. **Per-page live-classification cost.** `classify_recovery` runs one or more `git rev-parse`/ref-existence checks per row on a page — bounded by the clamped page size, not the table, but a page-size number has to be chosen and justified at implementation time; I decline to invent one here (see Corrections — A's original draft cited a wrong line number and an invented value for exactly this).

3. **Signature-touching change.** `recovers: Option<OperationId>` on `admit()`/`OperationStatus` needs every existing call site audited (including `durable.rs`'s own test helper constructing `OperationStatus` directly) — a plain struct rather than a builder makes a forgotten site a compile error, which works in this design's favor.
4. **Recovery-ref accumulation is surfaced, not solved, by this feature.** No deletion path for `refs/git-vista/recovery/<id>` exists in `durable.rs` today, and the SQLite table has no TTL (`insert_or_update` never deletes). Making operations browsable makes this accumulation visible for the first time rather than causing it — flagged for Tom below, not silently fixed.
5. **The `RepositoryToken`→path resolution path is specified, not yet proven.** `Catalog::resolve` takes a `WorktreeId` (a `git_vista_core::identity` UUID newtype); the durable row stores a `WorktreeToken` (a `git-vista-protocol` newtype). Whether the conversion between them is already a solved, reusable function or new wiring is something I did not fully trace — stated honestly rather than assumed, per "never claim a capability the code does not have."

Invariant table

Split three ways, because an invariant with no stated mutation is not verifiable, and a compile-time property is not the same kind of proof as a red test — claiming otherwise would be exactly the overclaim ADR 0058 exists to catch.

Compile-time — proven by the type checker, not by `mutation_check`

Invariant	How it's proven	Why not mutation-provable
A `RecoveryClass` other than `Offered` never carries a constructible `UndoAction`.	`undo: UndoAction` lives only inside the `Offered` variant's own fields, not a struct-level shared field. Hoisting it out to make a non-`Offered` arm carry one is a shape change the compiler enforces at every construction site and every match.	Breaking this requires editing the enum's shape, which fails the build before any test runs — `mutation_check` clones and mutates *code*, but a shape change here is not a localized `old_string`/`new_string` patch against one function's logic; it changes what every call site is allowed to do. Marked here as a design claim, to be re-examined once the enum is real code and a genuine one-line mutation against it can be attempted.

Mutation-provable today, against code that already exists

Invariant	Mutation	Where
A terminal operation's outcome (including a refusal) is durably persisted before the client's request returns — this is the actual load-bearing fact the whole Decision rests on.	Delete the second `crate::durable::persist(durable_key, terminal.clone())` call (`planner.rs:264`). A test that runs an operation to a refusal (e.g. a deliberately stale plan) and then reads the row from a fresh connection to the same file should find no terminal row, or a stale non-terminal one, once this call is removed.	`planner.rs:264`
The recovery-ref pin is written before the destructive command runs, inside the mutation guard.	Move `pin_recovery`'s call site (`planner.rs:341-344`) from before `crate::operations::stage(OperationStage::Executing);` `execute(...)` to after `execute` returns. A test that force-deletes a branch concurrently with a `git gc` sweep and asserts the pin still resolves should go red.	`planner.rs:341-344`

Invariant	Mutation	Where
A live check that cannot establish an answer must not be read as "safe to offer" (the precedent this design's `CheckFailed` arm generalizes).	In `revert_offer_established`, flip `.map(|conflicts| !conflicts).unwrap_or(false)` to `unwrap_or(true)`. A test asserting "a `git merge-tree` spawn failure yields no revert offer" should go red once the fail-open default is substituted.	`activity.rs:360` (the existing precedent this feature's `CheckFailed` logic must match, not new code)

Specified, not yet provable — the code does not exist yet

Invariant	Intended mutation (to run once the code lands)	Status
`POST /api/operations/{id}/recover` refuses (409) whenever live re-classification does not equal the client-submitted `UndoAction`, even if the submitted action was genuinely offered a moment earlier.	Delete the equality comparison in step 3 of `recover_operation` so the handler executes `claimed` unconditionally once the class is merely `Offered`-shaped. A test that reads an `Offered` class, invalidates it (moves the branch), then POSTs the stale action and asserts 409 should flip to 200-and-executed.	Not provable until `recover_operation` exists.

Invariant	Intended mutation (to run once the code lands)	Status
`list_operations` never rewrites a terminal row — the same non-mutating contract `recover_blocking`'s doc reserves for startup only.	Have `list_operations`'s query also close out non-terminal rows (mirroring `recover_blocking`'s own logic). A concurrency test inserting a still-`Running` row via a private connection, then calling `list_operations`, should find it byte-identical before/after, and go red if the close-out logic leaks in.	Not provable until `list_operations` exists.
A `RecoveryClass::CheckFailed { RepositoryNotRegistered }` is produced, never a false `Expired`, when the row's token doesn't resolve in the catalog.	In `classify_recovery`, change the branch that handles `Catalog::resolve` returning `None` to instead classify as `Expired { PinMissing }`. A test with a deregistered repository's history row should assert `CheckFailed`, not `Expired`, and go red under the substitution.	Not provable until `classify_recovery` exists; the gap itself is real today (Consequences #5, Model).

Genuinely Tom's call

- **Retention/archival for the operations table**, now that it becomes user-browsable and has no TTL (`insert_or_update` never deletes, unlike the in-memory `Registry`'s `MAX_RECORDS=256`/`RECORD_TTL_SECS=3600`, `operations.rs:68,73`). Unbounded forever, or a policy?
- **Recovery-ref cleanup.** I found no deletion path for `refs/git-vista/recovery/<id>` anywhere in `durable.rs`. Stating that plainly, not proposing a fix — whether/when to clean these up is a product decision about how long "undo" should remain offered, not an engineering default.
- **Whether extending `UndoAction` to close the `KnownNotWired` gap (`DeleteCreatedBranch`, `RecreateTag`, `DeleteCreatedTag`, `CheckoutPrevious`) is in #78's scope or a fast-follow.** The type-system safety property holds either way; this only decides how many rows have a working button on day one.
- **Single-repository vs. cross-repository history scope.** The design defaults to the currently-selected repository to match the rest of the UI, but this is an assumption, not something grounded in an existing multi-repo history surface — and either choice must handle the `RepositoryNotRegistered` gap above.

Decided here, not punted

Source of truth (SQLite, not reflog); keyset pagination on `(accepted_at, id)` (`mint_id` is CSPRNG, not time-ordered); classification computed live and never served from the stored `recovery_json`; server-side re-derivation plus an equality gate at the write boundary as the actual enforcement point; `recovers_operation` answered by read-time lookup rather than a mutable flag on the original row; stash out of scope by construction, forward-compatible via `KnownNotWired`.

Corrections to the source designs

Task item 4 asked what a design assumed the grounding did not confirm. Independently verified against the tree, not merely restated:

- **`RecoveryStrategy` has ten variants, not nine as Approach A stated** (`plan.rs:994-1071`: `NotNeeded`, `ResetRef`, `RecreateBranch`, `DeleteCreatedBranch`, `RecreateTag`, `DeleteCreatedTag`, `CheckoutPrevious`, `RevertCommit`, `RecoverableIfStaged`, `Irrecoverable`). A's four-variant `KnownNotWired` gap list itself is correct; only the total count was wrong.
- **Approach A's "MAX_LIMIT convention, activity.rs:236, e.g. 20-50" is wrong twice over** — the constant is at `activity.rs:42`, not 236, and its value is `500`, not a number in the 20-50 range. A invented a page-size figure it presented as an existing convention; this spec declines to repeat the error and leaves the actual page-size limit for implementation to set and justify.
- **Approach B's claim of a new `crates/git-vista-protocol/src/history.rs` module is false** — the file exists today, exporting `HistoryFrame`/`HistoryPage` for the unrelated M1.10 paged-commit-graph feature (#63). Documented in full under Alternatives considered.
- **Approach B's `ORIG_HEAD`/stash negative findings were re-run by me, not merely trusted** — both confirmed independently (see Decision, graft #3).
- **B's gc-expiry numbers are environment-verified** (`man` output on this box, git 2.43.0), **not code-verified** — no repository config was read to confirm no override exists beyond "none configured on this box today." Labeled as such rather than presented as a codebase fact.
- **A genuine open gap neither source design named:** a durable history row's `RepositoryToken`/`WorktreeToken` can point at a repository the in-memory `Catalog` no longer holds (`Catalog::resolve` returns `None` for any id it doesn't have, confirmed by its own test `resolve_returns_none_for_an_unknown_id`, `catalog.rs:547-557`) — moved, deregistered, or simply not yet rescanned since a restart. `classify_recovery` has no path to run git against in that case. Given its own `RecoveryClass::CheckFailed` arm above rather than silently folding into `Expired`.

Signed: max · 2026-08-18